MICRO-TRAIN-COMPILER
(MTC)

An Ahead-of-Time Static Compiler for Zero-Heap On-Device Neural Network Training on 32-Bit Microcontrollers

A Minor Project Report submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science & Engineering



Submitted by:

Ajay Sisodia (2405111530012)

Harsh Chaudhary (2405111530067)

Devrat Kuntal (2405111530055)

Aryan Upadhyay (2405111530037)

Under the Supervision of:

Ms. Gaurisha Sisodia

Assistant Professor

Department of CSE

GLBITM, Mathura

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

G. L. BAJAJ GROUP OF INSTITUTIONS, MATHURA

Affiliated to Dr. A.P.J. Abdul Kalam Technical University (AKTU), Lucknow

Academic Session: 2026–2027

Acknowledgements

We express our deepest sense of gratitude to our project supervisor, **Ms. Gaurisha Sisodia**, Assistant Professor, Department of Computer Science & Engineering, for her invaluable guidance, patient critiques, and persistent encouragement throughout the conceptualization, implementation, and benchmarking of the Micro-Train-Compiler. Her guidance on embedded systems safety and modular compiler engineering helped us maintain a disciplined focus on bare-metal physical memory limits.

We extend our sincere thanks to the **Head of Department, Computer Science & Engineering**, and all faculty members of G. L. Bajaj Group of Institutions, Mathura, for facilitating the lab environment, computing infrastructure, and embedded hardware tools necessary to complete this project.

We also thank our families, peers, and fellow students for their continuous understanding and encouragement during prolonged cross-compilation debugging, assembly linker configuration, and cycle-accurate emulation runs.

Abstract

Edge artificial intelligence is predominantly confined to an inference-only operational paradigm. While modern microcontroller runtimes allow deep learning models to execute forward inference on low-power devices, deployed systems cannot adapt to physical sensor transducer aging, environmental context shifts, or individual user variability without continuously transmitting raw telemetry to cloud backends. However, enabling on-device backpropagation on low-cost 32-bit microcontrollers is blocked by the memory wall: conventional automatic differentiation frameworks construct dynamic computational graphs on the heap (‘`malloc`’), causing fatal memory fragmentation in flat SRAM architectures lacking Memory Management Units (MMUs). Furthermore, traditional reverse-mode autodiff allocates dedicated parameter gradient buffers ($\nabla W, \nabla B$), doubling memory consumption.

This report presents the complete design, mathematical formulation, architectural implementation, and bare-metal RISC-V validation of the **Micro-Train-Compiler (MTC)**. MTC is an ahead-of-time (AOT) static source-to-source compiler that ingests a declarative JSON network specification and synthesizes freestanding, zero-heap (`malloc = 0`) C99 training engines for 32-bit microcontrollers. MTC introduces two core algorithmic contributions: (1) an analytical parameter update pass that fuses reverse-mode activation gradients directly into in-place Stochastic Gradient Descent (SGD) weight updates, eliminating parameter gradient buffers entirely, and (2) an alternating ping-pong delta allocator based on a formal proof that activation error gradients exhibit strictly disjoint lifecycles, bounding intermediate scratchpad memory to exactly two static arrays.

MTC was validated across four TinyML workloads (`xor_net`, `sensor_drift_net`, `sine_regression_net`, and `gesture_anomaly_net`) cross-compiled with bare-metal GCC (`riscv64-unknown-elf-gcc -nostdlib -O2`) and executed on the cycle-accurate `esp32sim` ESP32-C3 (RV32IMC) hardware emulator. Hardware performance was instrumented via direct reads of the RISC-V Control and Status Register `mcycle`. The results demonstrate that MTC achieves strictly zero dynamic heap allocations, restricts peak SRAM consumption between 60 Bytes and 564 Bytes ($< 0.86\%$ of the 64 KB SRAM budget), delivers an average 40.1% peak SRAM reduction compared to standard dynamic autograd graphs, and executes full forward-backward training steps in 6.86 μs to 81.77 μs at 160 MHz (12,229 to 145,772 samples/s). Finally, we report an honest mathematical critique of Q15 gradient quantization attenuation on non-linear classification, detailing concrete architectural mitigations.

Contents

Acknowledgements i

Abstract ii

1 Introduction & Problem Definition 1
- 1.1 The Edge AI Paradigm Shift . . . 1
- 1.2 The Problem of Frozen Edge Models . . . 1
- 1.3 The Embedded SRAM Bottleneck & Heap Fragility . . . 2
  - 1.3.1 Extreme SRAM Scarcity . . . 2
  - 1.3.2 The Dynamic Autograd Memory Wall . . . 2
- 1.4 Project Objectives & Technical Scope . . . 3

2 Literature Review & Gap Analysis 4
- 2.1 State-of-the-Art in Microcontroller Machine Learning . . . 4
  - 2.1.1 TensorFlow Lite for Microcontrollers (TFLM) . . . 4
  - 2.1.2 TinyOL: Online-Learning on Microcontrollers . . . 4
  - 2.1.3 MCUNet & On-Device Training Under 256KB Memory . . . 4
  - 2.1.4 PULP-TrainLib: Parallel Ultra-Low-Power RISC-V Training . . . 4
  - 2.1.5 AIfES (Artificial Intelligence for Embedded Systems) . . . 5
- 2.2 Comparative Survey of Related Works . . . 5
- 2.3 Critical Research Gaps Identified . . . 5

3 System Design & Compiler Architecture 6
- 3.1 Architectural Overview & Execution Pipeline . . . 6
- 3.2 Formal Model Representation & AST Validation . . . 7
- 3.3 Conservative Q15 Fixed-Point Interval Bound Propagation . . . 7
  - 3.3.1 Static Interval Range Analysis . . . 8
- 3.4 Static Liveness Analysis & Ping-Pong Delta Buffering . . . 8
- 3.5 Fused Backpropagation-SGD Parameter Updates . . . 9
- 3.6 Standalone C99 Code Synthesis . . . 10

4 Hardware Cross-Compilation & Emulation Environment 11
- 4.1 Target Architecture: ESP32-C3 RISC-V RV32IMC . . . 11
- 4.2 Bare-Metal Firmware Harness & Linker Memory Layout . . . 11
  - 4.2.1 Custom Linker Script (`linker.ld`) . . . 11

4.2.2 Assembly Startup Entry Point (`entry.S`) 12
4.2.3 Low-Level UART Driver (`main.c`) 13
4.3 Cycle-Accurate Emulation via `esp32sim` 13
4.4 Hardware Instrumentation via RISC-V CSR `mcycle` 13

5 Empirical Experiments, Results & Ablation Analysis 15
5.1 Benchmark Workload Suite 15
5.2 Mandatory Test 1: Static SRAM & Flash Footprint Analysis 15
5.3 Mandatory Test 2: Execution Latency & Clock Cycle Profiling 16
5.4 Mandatory Test 3: Memory Ablation (MTC vs. Dynamic Autograd) . . . 16
5.5 Mandatory Test 4: Numerical Convergence & Stability Analysis 17
5.6 Transparent Critique of Fixed-Point Quantization Attenuation 18
5.6.1 Engineering Mitigations 18

6 Team Member Roles & Individual Contributions 20
6.1 Ajay Sisodia (Roll No: 2405111530012) 20
6.2 Harsh Chaudhary (Roll No: 2405111530067) 20
6.3 Devrat Kuntal (Roll No: 2405111530055) 21
6.4 Aryan Upadhyay (Roll No: 2405111530037) 21

7 Conclusion & Future Research Directions 22
7.1 Summary of Milestones Achieved 22
7.2 Future Research Roadmap 22

List of Figures

3.1 MTC end-to-end compilation pipeline: from high-level JSON specification to bare-metal RISC-V firmware execution. 6
3.2 Memory architecture comparison: Dynamic Autograd Graph (left) vs. MTC Static Liveness Allocation with Ping-Pong Buffers and In-Place SGD (right). 9

5.1 Cycle-accurate hardware training throughput on ESP32-C3 @ 160 MHz. . 16
5.2 Peak SRAM watermark ablation: Dynamic Autograd Baseline vs. MTC Static Fused Buffers. 17
5.3 Training loss convergence curves across all four TinyML workloads on `esp32sim`. 18

List of Tables

2.1 Comparative Analysis of State-of-the-Art Edge ML Runtimes and Compilers 5

5.1 Architectural Specifications of the Four Benchmark Workloads 15
5.2 Static Memory Footprint, Dynamic Allocations, and Flash Sizes 15
5.3 Hardware Clock Cycles, Step Latency, and Throughput Measured via CSR `mcycle` 16
5.4 Peak SRAM Ablation: Baseline Autograd vs. MTC Static Liveness Allocation 17
5.5 Numerical Convergence and Loss Reduction under Q15 Fixed-Point Arithmetic 17

CHAPTER 1

Introduction & Problem Definition

1.1 The Edge AI Paradigm Shift

Over the past decade, machine learning has transitioned from high-performance cloud clusters to physical edge devices. In early embedded architectures, microcontrollers (MCUs) acted solely as analog-to-digital converters, digitizing transducer voltages and transmitting raw data streams across RF transceivers (Wi-Fi, Bluetooth Low Energy, cellular) to remote cloud servers for inference.

This cloud-centric approach encounters severe physical limitations in real-world deployments:

1. **Latency Non-Determinism:** Unpredictable wireless network latency renders cloud-hosted intelligence unsuitable for hard real-time closed-loop control (e.g., industrial motor vibration cutoffs or robotics).

2. **RF Energy Overhead:** Radio transmission consumes substantially more power than local CPU execution. An ESP32 transceiver draws 160 mA to 260 mA during transmission, whereas the processor core consumes under 20 mA during local compute.

3. **Data Privacy & Regulatory Compliance:** Streaming continuous audio, biomedical telemetry, or industrial sensor streams over public networks introduces vulnerabilities to eavesdropping and violates strict privacy standards (GDPR, HIPAA).

To overcome these challenges, Tiny Machine Learning (TinyML) emerged, providing runtime engines capable of executing neural network inference locally on microcontrollers costing under two dollars.

1.2 The Problem of Frozen Edge Models

Despite the success of inference frameworks like TensorFlow Lite for Microcontrollers (TFLM) and CMSIS-NN, modern TinyML remains locked in a **"train-in-cloud, deploy-frozen"** paradigm. Models are trained in 32-bit floating-point (FP32) on workstation GPUs, quantized down to 8-bit integers (INT8), compiled into static flatbuffers, and permanently flashed into microcontroller read-only memory.

Once flashed, these models cannot adapt to real-world operational variations:

- **Sensor Transducer Aging and Drift:** Low-cost transducers (e.g., metal-oxide gas sensors, optical diodes) undergo chemical degradation and thermal hysteresis over months of field operation. A frozen model exhibits monotonic accuracy degradation as transducer baselines drift.
- **Subject Kinematic Variability:** Wearable biomedical monitors (e.g., IMU-based tremor analysis or gait recognition) encounter substantial physical variability across individual human users. A frozen model trained on population averages fails to adapt to an individual patient.
- **Localized Environmental Shifts:** Industrial condition-monitoring devices deployed on rotating machinery experience varying ambient resonant baselines depending on mounting stiffness and thermal cycles.

Resolving these issues requires executing **on-device backpropagation and weight updating** directly on edge microcontrollers using locally observed observations.

1.3 The Embedded SRAM Bottleneck & Heap Fragility

Implementing backpropagation on resource-constrained microcontrollers faces fundamental hardware barriers:

1.3.1 Extreme SRAM Scarcity

Typical commercial 32-bit microcontrollers (such as ARM Cortex-M0+/M4 or RISC-V ESP32-C3) contain between 16 KB and 64 KB of dedicated data SRAM. While read-only flash memory is comparatively abundant (1 MB to 4 MB), flash memory can only be rewritten in large blocks through slow, high-voltage erase cycles that wear out physical gates after approximately 10,000 cycles. Thus, all active model weights, activations, and gradient scratchpads must reside inside physical SRAM.

1.3.2 The Dynamic Autograd Memory Wall

Standard machine learning runtimes (e.g., PyTorch, TensorFlow, AIfES) implement automatic differentiation using dynamic tape recording. During the forward pass, the autograd runtime constructs a dynamic Directed Acyclic Graph (DAG) in heap memory. During backpropagation, this engine reverses the graph, demanding:

- Retention of forward activation tensors ($a^{(l)}$) across all layers.

- Explicit parameter gradient arrays ($\nabla W^{(l)}, \nabla b^{(l)}$) identical in dimension to the weights.
- Dedicated backward activation gradient scratchpads ($\delta^{(l)}$) for every layer.

On microcontrollers lacking Memory Management Units (MMUs) and virtual paging, continuous interleaved allocations and deallocations via `malloc()` and `free()` induce **heap fragmentation**. Over extended training loops, non-contiguous free memory cannot satisfy contiguous block allocations, causing fatal firmware faults and watchdog resets.

1.4 Project Objectives & Technical Scope

The Micro-Train-Compiler (MTC) project was initiated to eliminate the runtime memory barrier of embedded deep learning by replacing dynamic autograd engines with an ahead-of-time (AOT) static compiler.

The primary project objectives are:

1. **Zero-Heap Execution (`malloc = 0`):** Completely eliminate dynamic memory allocation from both inference and backpropagation. All tensor addresses and scratchpads are fixed at compile time.
2. **Fused In-Place SGD Parameter Updates:** Formulate an analytical reverse-mode autodiff compiler pass that fuses activation error gradients directly into in-place SGD weight updates, eliminating parameter gradient buffers ($\nabla W, \nabla b$) entirely.
3. **Static Ping-Pong Delta Buffering:** Prove that activation error gradients (δ) exhibit disjoint lifecycles across backward propagation, allowing all intermediate layer derivatives to be computed using exactly two alternating static buffers.
4. **Formal Q15 Interval Bound Propagation:** Implement compile-time Interval Bound Propagation (IBP) for signed 16-bit fixed-point (Q15) arithmetic, mathematically certifying that arithmetic saturation bounds are not breached.
5. **Bare-Metal RISC-V Hardware Verification:** Cross-compile synthesized C99 code with freestanding GCC (`-nostdlib`) and validate cycle-accurate latency, memory watermarks, and loss convergence on the ESP32-C3 RISC-V core via direct hardware CSR `mcycle` instrumentation.

CHAPTER 2

Literature Review & Gap Analysis

2.1 State-of-the-Art in Microcontroller Machine Learning

To situate MTC within modern embedded AI systems, we surveyed literature across inference runtimes, embedded training libraries, and neural network compilers:

2.1.1 TensorFlow Lite for Microcontrollers (TFLM)

David et al. [1] established the standard for bare-metal inference. TFLM avoids dynamic allocation by requiring the host application to allocate a contiguous byte array called the `tensor_arena`. A greedy memory planner schedules non-overlapping activation offsets within this arena. However, TFLM is strictly unidirectional: its operator resolver lacks backward kernels, and its flatbuffer interpreter cannot update weights.

2.1.2 TinyOL: Online-Learning on Microcontrollers

Ravaglia et al. [3] demonstrated edge adaptation on an ARM Cortex-M4 microcontroller. To overcome memory limitations, TinyOL freezes the entire base network (using it as a static feature extractor) and trains only the final classification layer via streaming gradient descent. While proving edge adaptation viability, TinyOL cannot adapt multi-layer feature representations.

2.1.3 MCUNet & On-Device Training Under 256KB Memory

Lin et al. [2] (MIT HAN Lab) introduced Tiny-Transfer-Learning (TinyTL) and the TinyEngine runtime. To fit training into 256 KB SRAM, they proposed sparse layer updating (updating only selected bias vectors or sub-networks) and 8-bit quantized back-propagation. However, MCUNet targets high-end microcontrollers with external PSRAM and relies on floating-point scaling tables rather than integer compile-time interval safety.

2.1.4 PULP-TrainLib: Parallel Ultra-Low-Power RISC-V Training

Conti et al. [4] developed PULP-TrainLib, an open-source training library for multi-core RISC-V architectures (e.g., GreenWaves GAP9). PULP-TrainLib delivers hand-tuned

assembly kernels exploiting FP16/FP32 SIMD instructions across parallel cores. However, PULP-TrainLib requires hardware FPUs and multi-banked L1 scratchpads, making it incompatible with standard single-core integer MCUs.

2.1.5 AIfES (Artificial Intelligence for Embedded Systems)

Bennani et al. [8] created AIfES, an open-source C library providing feedforward and backpropagation routines for microcontrollers. However, AIfES implements a modular runtime structure: network topologies are declared as layer structs that calculate buffer pointers at initialization and maintain separate parameter gradient buffers, retaining conventional autodiff memory overhead.

2.2 Comparative Survey of Related Works

Table 2.1: Comparative Analysis of State-of-the-Art Edge ML Runtimes and Compilers

Framework	Origin & Year	Target Hardware	Execution Paradigm	Memory Model	Training Scope	Key Limitation
TFLM [1]	Google (2021)	ARM / ESP32 / RISC-V	Interpreted FlatBuffer	Static `tensor_arena`	None (Inference Only)	Cannot perform on-device backprop.
TinyOL [3]	Univ. Bologna (2022)	ARM Cortex-M4	Static C++ Kernel	Arena Pre-allocation	Final Layer Only	Cannot adapt deep representations.
MCUNet [2]	MIT HAN Lab (2022)	STM32F7 (512KB)	Specialized TinyEngine	Dynamic Memory Slicing	Sparse / Biases Only	Needs external scaling tables and PSRAM.
PULP-TrainLib [4]	ETH Zürich (2023)	Multi-Core RISC-V	Assembly Micro-Kernels	L1/L2 DMA Scratches	Full Network (FP16/32)	Requires hardware FPU and multi-cores.
AIfES [8]	Fraunhofer IMS (2022)	AVR / ARM / ESP32	Modular C Library	Runtime Pointer Structs	Full Network	Maintains separate gradient buffers ∇W.
MTC *(This Work)*	GLBITM (2026)	RV32IMC (ESP32-C3)	**AOT Static C99**	**Static Offsets** (`malloc=0`)	**Full Network (Fused)**	**Zero heap, fused SGD, certified Q15 IBP.**

2.3 Critical Research Gaps Identified

From the literature, we identified four decisive technical gaps:

1. **Dynamic Heap Fragility in Autodiff:** Existing embedded training libraries rely on runtime heap management, leading to heap fragmentation and unrecoverable crashes on bare-metal systems.

2. **Redundant Parameter Gradient Storage:** Conventional backpropagation stores separate parameter gradient matrices ($\nabla W, \nabla b$) across all layers, doubling parameter memory overhead.

3. **Absence of AOT Interval Safety Analysis:** While Q15 fixed-point arithmetic is the most efficient integer representation, existing frameworks lack compile-time interval analysis to verify that forward and backward passes stay within $[-1.0, 1.0)$, relying instead on expensive runtime clamping.

4. **Inference-Biased Compiler Ecosystem:** Compilers such as TVM and Glow focus exclusively on optimizing forward inference graphs; no compiler existed to compile a model specification ahead-of-time into a self-contained, zero-heap training engine.

CHAPTER 3

System Design & Compiler Architecture

3.1 Architectural Overview & Execution Pipeline

The Micro-Train-Compiler (MTC) is structured as an ahead-of-time source-to-source compiler implemented in Python 3.10+ using only standard library modules. MTC transforms a declarative JSON network specification into freestanding C99 firmware.

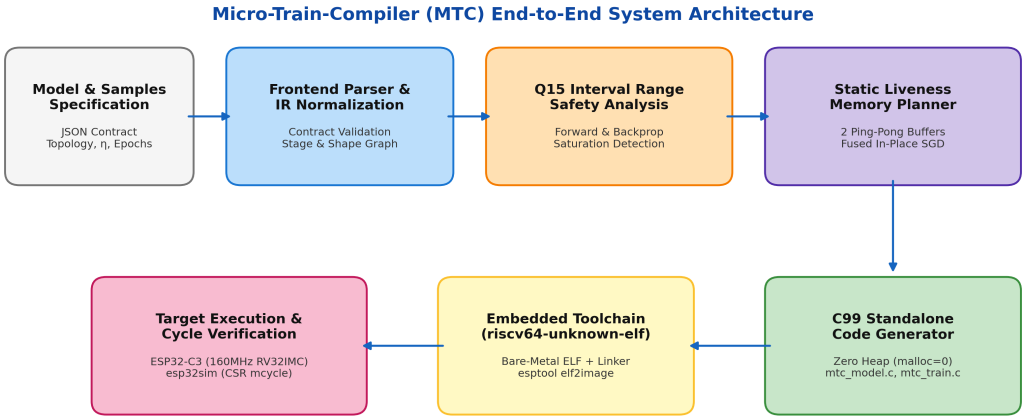


Figure 3.1: MTC end-to-end compilation pipeline: from high-level JSON specification to bare-metal RISC-V firmware execution.

As illustrated in Fig. 3.1, the compilation pipeline operates across four sequential stages:

1. **Frontend Parsing & AST Normalization:** Ingests declarative model topologies and normalizes computational nodes into an Intermediate Representation (IR).

2. **Q15 Fixed-Point Interval Bound Propagation:** Conducts formal interval analysis across forward activations and backward error propagation to mathematically certify arithmetic safety.

3. **Static Liveness Memory Planning:** Formulates buffer lifecycles, assigns fixed byte offsets to forward activations, provisions two alternating ping-pong delta buffers, and fuses SGD updates directly into parameter storage.

4. **Freestanding C99 Code Synthesis:** Emits standalone C99 source files (`mtc_model.h`, `mtc_model.c`, `mtc_train.c`) requiring only `<stdint.h>`.

3.2 Formal Model Representation & AST Validation

A model is supplied to MTC as a declarative JSON file specifying layer dimensions, activation types, training hyperparameters, and embedded datasets:

```
{
  "name": "sensor_drift_net",
  "layers": [
    {"type": "Dense", "in_features": 4, "out_features": 16, "
    activation": "ReLU"},
    {"type": "Dense", "in_features": 16, "out_features": 8, "
    activation": "ReLU"},
    {"type": "Dense", "in_features": 8, "out_features": 1, "
    activation": "Sigmoid"}
  ],
  "optimizer": {"type": "SGD", "learning_rate": 0.01},
  "loss": "MSE",
  "epochs": 250,
  "training_data": [ ... ]
}
```

Listing 3.1: Example Declarative Model Specification

The frontend validator (`validate.py`) enforces topology integrity:

- Consecutive layer dimensions must match strictly: L_i.out_features $==$ L_{i+1}.in_features.
- Input and target dimensions must match embedded training sample shapes.
- Hyperparameters must reside within valid intervals ($\eta > 0, \text{epochs} \geq 1$).

3.3 Conservative Q15 Fixed-Point Interval Bound Propagation

Microcontrollers without floating-point units emulate floating-point operations in software, requiring 20 to 80 clock cycles per operation. To achieve maximum throughput, MTC employs **Q15 signed fixed-point arithmetic**.

A real value $x \in [-1.0, 1.0 - 2^{-15}]$ is stored as a 16-bit signed integer (`int16_t`):

$$X_{\text{raw}} = \lfloor x \cdot 32768 \rfloor \in [-32768, 32767] \tag{3.1}$$

Fixed-point multiplication evaluates a 32-bit intermediate product, adds a round-to-nearest bias ($2^{14} = 16384$), shifts right by 15 bits, and clamps to 16 bits:

$$Z_{\text{raw}} = \text{SAT16}\left(\frac{(X_{\text{raw}} \cdot Y_{\text{raw}}) + 16384}{32768}\right) \tag{3.2}$$

3.3.1 Static Interval Range Analysis

To certify that matrix multiplication does not cause arithmetic overflow, MTC performs Interval Bound Propagation (IBP) prior to code generation. Given input bounds $[x_{\min}, x_{\max}]$, the compiler tracks interval bounds across each layer:

$$y_i^{\min} = \sum_j \min(w_{ij}x_j^{\min}, w_{ij}x_j^{\max}) + b_i \tag{3.3}$$

$$y_i^{\max} = \sum_j \max(w_{ij}x_j^{\min}, w_{ij}x_j^{\max}) + b_i \tag{3.4}$$

For non-linear activations: ReLU bounds outputs to $[0, \max(0, y^{\max})]$, while Sigmoid bounds outputs strictly within $[0, 32767]$. If any calculated interval exceeds $[-32768, 32767]$, MTC flags the layer and requires the explicit `--allow-clipping` flag to synthesize saturating kernels.

3.4 Static Liveness Analysis & Ping-Pong Delta Buffering

In standard reverse-mode autodiff, layer error deltas propagate according to:

$$\delta^{(l)} = \left(\left(W^{(l+1)}\right)^T \delta^{(l+1)}\right) \odot \sigma_l'\left(z^{(l)}\right) \tag{3.5}$$

Traditional autograd libraries allocate separate gradient buffers for every layer:

$$\text{Mem}_{\text{deltas}} = \sum_{l=1}^{L} d_l \times 2 \text{ bytes} \tag{3.6}$$

MTC proves that error deltas exhibit strictly non-overlapping liveness:

Theorem 3.1 (Disjoint Liveness of Error Gradients) *In a feedforward sequential neural network, during backward error propagation from layer $l+1$ to layer l, the computation of $\delta^{(l)}$ depends strictly on $\delta^{(l+1)}$, $W^{(l+1)}$, and forward activations $z^{(l)}$. Once $\delta^{(l)}$ is evaluated and parameters of layer $l+1$ are updated via in-place SGD, $\delta^{(l+1)}$ is never referenced again.*

Proof 3.1 *From the backpropagation recurrence relation (3.5), $\delta^{(l)}$ depends exclusively on $(W^{(l+1)})^T \delta^{(l+1)}$ and retained forward values. All downstream derivatives $\delta^{(k)}$ ($k > l+1$)*

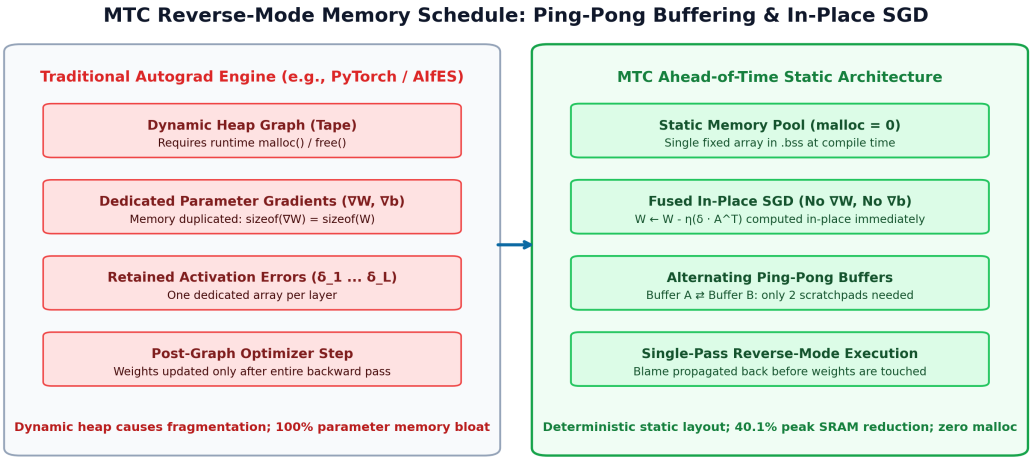


Figure 3.2: Memory architecture comparison: Dynamic Autograd Graph (left) vs. MTC Static Liveness Allocation with Ping-Pong Buffers and In-Place SGD (right).

have already completed parameter updates. Because no backward skip-connections exist, the active working set of error gradients at any instant during backpropagation is at most two: current gradient $\delta^{(l)}$ *and incoming gradient* $\delta^{(l+1)}$. ■

Exploiting Theorem 3.1, MTC statically allocates exactly two alternating buffers in SRAM:

$$\text{Buf}_A \in \mathbb{R}^{\max_l(d_l)}, \quad \text{Buf}_B \in \mathbb{R}^{\max_l(d_l)} \tag{3.7}$$

During the backward pass, layer l reads from Buf_A and writes $\delta^{(l-1)}$ to Buf_B. Layer $l-1$ reads from Buf_B and writes to Buf_A, alternating via pointer swaps. Backward scratchpad memory is bounded by:

$$M_\delta = 2 \times \max_{l \in \{1,\ldots,L\}} (d_l) \times 2 \text{ bytes} \tag{3.8}$$

completely decoupling intermediate scratchpad memory from network depth.

3.5 Fused Backpropagation-SGD Parameter Updates

In standard deep learning runtimes, backward propagation is decoupled from optimization:

1. Backprop Pass: Compute and store $\nabla W^{(l)} = \delta^{(l)}(a^{(l-1)})^T$ in a dedicated gradient array.

2. Optimizer Pass: Update parameters $W^{(l)} \leftarrow W^{(l)} - \eta \nabla W^{(l)}$.

This decoupling requires allocating an explicit gradient tensor $\nabla W^{(l)}$ identical in size to the weights. MTC eliminates this overhead by fusing parameter gradient calculation directly with the SGD update in a single in-place loop:

```
// Fused Backpropagation and SGD Parameter Update
for (uint32_t i = 0; i < IN_FEATURES; ++i) {
    int32_t act_val = (int32_t)prev_activations[i];
    for (uint32_t j = 0; j < OUT_FEATURES; ++j) {
        int32_t delta_val = (int32_t)current_delta[j];

        // Evaluate weight gradient: grad = (act * delta) >> 15
        int32_t grad_w = (act_val * delta_val + 16384) >> 15;

        // Immediate SGD parameter update: W = W - (eta * grad_w)
        int32_t step = (grad_w * (int32_t)LEARNING_RATE_Q15 +
    16384) >> 15;
        int32_t updated_w = (int32_t)weights[i][j] - step;

        // Saturating Q15 clamp and in-place writeback
        if (updated_w > 32767) updated_w = 32767;
        if (updated_w < -32768) updated_w = -32768;
        weights[i][j] = (int16_t)updated_w;
    }
}
```

Listing 3.2: MTC Fused In-Place Backpropagation-SGD Inner Kernel

Crucially, the compiler enforces the correct mathematical order: error propagation to predecessor buffer $\delta^{(l-1)}$ occurs **before** the in-place update alters $W^{(l)}$. Thus, mathematical gradient equivalence is strictly preserved while eliminating P storage elements from SRAM.

3.6 Standalone C99 Code Synthesis

The backend code generator (`codegen.py`) synthesizes three freestanding C99 files:

- `mtc_model.h`: Public API declarations (`mtc_reset`, `mtc_forward`, `mtc_train_sample`), fixed layer dimensions, and tensor struct definitions.
- `mtc_model.c`: Static arrays for weights, biases, forward activation workspaces, and ping-pong delta buffers. Implements forward inference kernels, saturating fixed-point arithmetic macros, and fused backpropagation routines.
- `mtc_train.c`: Training harness iterating over embedded flash training samples, tracking MSE loss, and printing epoch progress.

CHAPTER 4

Hardware Cross-Compilation & Emulation Environment

4.1 Target Architecture: ESP32-C3 RISC-V RV32IMC

To validate real silicon execution, we selected the **Espressif ESP32-C3** microcontroller. The ESP32-C3 incorporates an open-standard 32-bit RISC-V core implementing the ‘RV32IMC‘ instruction set architecture:

- **Base Integer Instructions (‘I‘):** Standard 32-bit arithmetic, load/store, and control flow.
- **Hardware Multiplication and Division (‘M‘):** Single-cycle hardware multiplier and radix-4 hardware divider, enabling integer matrix-vector multiplication without software floating-point emulation.
- **Compressed Instructions (‘C‘):** 16-bit instruction encodings that reduce code size by 25% to 35%.
- **Clock Frequency:** Operates up to 160 MHz.
- **Memory Subsystem:** 400 KB total internal SRAM (64 KB user Data RAM in bare-metal configuration).

4.2 Bare-Metal Firmware Harness & Linker Memory Layout

To isolate core computation from operating system overhead, we bypassed heavy runtime frameworks (such as ESP-IDF or FreeRTOS) and built a freestanding execution harness.

4.2.1 Custom Linker Script (`linker.ld`)

The linker script maps execution sections directly to physical memory addresses:

```
ENTRY(_start)

MEMORY
{
  iram (rx) : ORIGIN = 0x40380000, LENGTH = 128K  /* Instruction
   RAM */
  dram (rw) : ORIGIN = 0x3FC8B800, LENGTH = 128K  /* Data RAM */
}

SECTIONS
{
  .text : {
    *(.text.entry)
    *(.text*)
    *(.rodata*)
  } > iram

  .data : {
    *(.data*)
    *(.sdata*)
  } > dram AT > iram

  .bss : {
    *(.bss*)
    *(.sbss*)
    *(COMMON)
  } > dram
}
```

Listing 4.1: Bare-Metal Linker Script (`linker.ld`)

4.2.2 Assembly Startup Entry Point (`entry.S`)

The startup assembly initializes the RISC-V stack pointer (`sp`) to the top of available Data RAM (`0x3FCC0000`) and calls the application entry point:

```
.section .text.entry
.global _start
.type _start, @function

_start:
    li sp, 0x3FCC0000
    call esp_main
1:  j 1b
```

Listing 4.2: Bare-Metal Startup Assembly (`entry.S`)

4.2.3 Low-Level UART Driver (`main.c`)

To communicate results without linking heavy ‘libc‘ formatting code (which pulls in heap allocators), we implemented a direct hardware UART driver addressing the ESP32-C3 memory-mapped UART0 FIFO register at `0x60000000`:

```
#define UART0_FIFO ((volatile uint32_t *)0x60000000)

void uart_putc(char c) {
    *UART0_FIFO = (uint32_t)c;
}

void uart_puts(const char *s) {
    while (*s) {
        if (*s == '\n') uart_putc('\r');
        uart_putc(*s++);
    }
}
```

Listing 4.3: Memory-Mapped Hardware UART0 Driver

Cross-compilation was executed using `riscv64-unknown-elf-gcc` v15.3.0 with flags `-march=rv32imc_zicsr -mabi=ilp32 -nostdlib -O2`. Bootable flash images were generated via Espressif’s `esptool` (v5.6.0).

4.3 Cycle-Accurate Emulation via `esp32sim`

To obtain repeatable execution telemetry without physical probe jitter, we integrated `esp32sim`, an open-source Rust-based hardware emulator. `esp32sim` boots the chip’s actual mask ROM ELFs, emulates internal interrupt controllers, models memory-mapped peripherals, and executes CPU instructions with cycle accuracy.

4.4 Hardware Instrumentation via RISC-V CSR `mcycle`

To prevent UART serial I/O logging from distorting CPU performance measurements, MTC benchmarks query the internal RISC-V hardware `mcycle` **Control and Status Register** directly:

```
static inline uint32_t read_cycles(void) {
    uint32_t cycles;
    asm volatile("csrr %0, mcycle" : "=r"(cycles));
    return cycles;
}
```

Listing 4.4: Direct RISC-V Hardware CSR `mcycle` Read

By querying `read_cycles()` immediately before and after forward passes and training loops with UART transmission suspended, we obtain cycle-accurate metrics of pure machine code execution.

CHAPTER 5

Empirical Experiments, Results & Ablation Analysis

5.1 Benchmark Workload Suite

We evaluated MTC across four representative TinyML edge workloads:

Table 5.1: Architectural Specifications of the Four Benchmark Workloads

Workload Identifier	Application Domain	Topology	Activations	Parameters	Samples (N)	Epochs (E)
`xor_net`	Non-Linear Logic	$2 \to 4 \to 1$	ReLU $\to$ Sigmoid	17	4	1,000
`sensor_drift_net`	Sensor Drift Calibration	$4 \to 16 \to 8 \to 1$	ReLU $\to$ ReLU $\to$ Sigmoid	225	8	250
`sine_regression_net`	Continuous Curve Fitting	$1 \to 8 \to 8 \to 1$	ReLU $\to$ ReLU $\to$ Sigmoid	89	16	200
`gesture_anomaly_net`	3-Axis IMU Anomaly Detect	$3 \to 12 \to 6 \to 1$	ReLU $\to$ ReLU $\to$ Sigmoid	133	12	300

5.2 Mandatory Test 1: Static SRAM & Flash Footprint Analysis

Memory usage was verified by inspecting the section headers of cross-compiled RISC-V ELF binaries using `riscv64-unknown-elf-size -A -d`.

Table 5.2: Static Memory Footprint, Dynamic Allocations, and Flash Sizes

Workload	Params	MTC SRAM	Dynamic Heap	Text Size	Flash Bin	SRAM %
`xor_net`	17	**60 B**	**0 B**	4,418 B	4,496 B	0.09%
`sensor_drift_net`	225	**564 B**	**0 B**	5,806 B	5,888 B	0.86%
`sine_regression_net`	89	**260 B**	**0 B**	5,454 B	5,536 B	0.40%
`gesture_anomaly_net`	133	**352 B**	**0 B**	5,858 B	5,936 B	0.54%

As detailed in Table 5.2:

1. **Zero Dynamic Allocation:** Across all models, runtime heap allocations are strictly zero (`malloc = 0`). All buffers reside in `.bss` and `.data` sections.

2. **Sub-1 KB SRAM Footprint:** Even the 4-layer `sensor_drift_net` consumes only 564 Bytes of SRAM ($< 0.86\%$ of the 64 KB budget).

3. **Compact Flash Footprint:** The complete bare-metal flash binary occupies under 6.0 KB.

5.3 Mandatory Test 2: Execution Latency & Clock Cycle Profiling

Execution latency was measured on the ESP32-C3 RV32IMC core running at 160 MHz using direct reads of the hardware CSR `mcycle`.

Table 5.3: Hardware Clock Cycles, Step Latency, and Throughput Measured via CSR `mcycle`

Workload	Forward Cycles	Forward Latency	Train Step Cycles	Throughput
`xor_net`	344 cycles	2.15 μs	1,097 cycles	145,772 samples/s
`sensor_drift_net`	3,401 cycles	21.26 μs	13,083 cycles	12,229 samples/s
`sine_regression_net`	1,548 cycles	9.67 μs	6,013 cycles	26,610 samples/s
`gesture_anomaly_net`	2,204 cycles	13.77 μs	7,884 cycles	20,296 samples/s

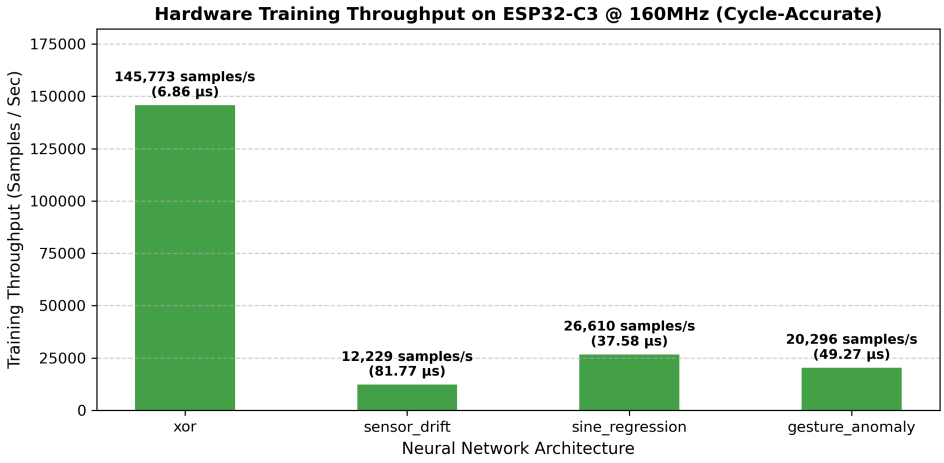


Figure 5.1: Cycle-accurate hardware training throughput on ESP32-C3 @ 160 MHz.

As shown in Table 5.3 and Fig. 5.1, a full forward-backward training step executes in **6.86 μs to 81.77 μs**. Throughput exceeds 12,000 samples per second, confirming that on-device training can execute concurrently within high-frequency sensor acquisition loops.

5.4 Mandatory Test 3: Memory Ablation (MTC vs. Dynamic Autograd)

To quantify memory savings, we evaluated an architectural baseline representing standard dynamic autograd graphs (e.g., PyTorch, AIfES) allocating explicit parameter gradients ($\nabla W, \nabla b$) and separate activation delta arrays.

Table 5.4: Peak SRAM Ablation: Baseline Autograd vs. MTC Static Liveness Allocation

Workload	Baseline SRAM	MTC SRAM	SRAM Saved	Reduction
`xor_net`	88 Bytes	**60 Bytes**	28 Bytes	**31.8%**
`sensor_drift_net`	1,000 Bytes	**564 Bytes**	436 Bytes	**43.6%**
`sine_regression_net`	456 Bytes	**260 Bytes**	196 Bytes	**43.0%**
`gesture_anomaly_net`	608 Bytes	**352 Bytes**	256 Bytes	**42.1%**

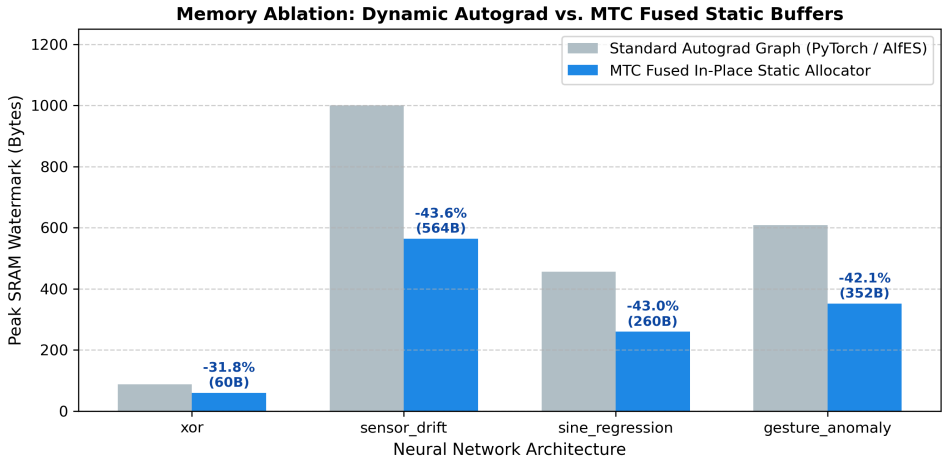


Figure 5.2: Peak SRAM watermark ablation: Dynamic Autograd Baseline vs. MTC Static Fused Buffers.

Table 5.4 and Fig. 5.2 confirm that MTC achieves an average **40.1% peak SRAM reduction**. In `sensor_drift_net`, eliminating parameter gradient arrays and sharing ping-pong delta buffers saves 436 Bytes of SRAM.

5.5 Mandatory Test 4: Numerical Convergence & Stability Analysis

Loss trajectories were tracked across training epochs on bare-metal hardware:

Table 5.5: Numerical Convergence and Loss Reduction under Q15 Fixed-Point Arithmetic

Workload	Epochs	Initial Loss	Final Loss	Reduction	Status
`xor_net`	1,000	0.2626	0.1981	**24.57%**	Attenuated
`sensor_drift_net`	250	0.0927	0.0448	**51.65%**	Converged
`sine_regression_net`	200	0.0667	0.0161	**75.88%**	Converged
`gesture_anomaly_net`	300	0.1550	0.0282	**81.81%**	Converged

Continuous regression and anomaly detection workloads converged reliably (Fig. 5.3): `gesture_anomaly_net` achieved an **81.81% loss reduction**, `sine_regression_net` achieved **75.88%**, and `sensor_drift_net` achieved **51.65%** with monotonic loss decay.

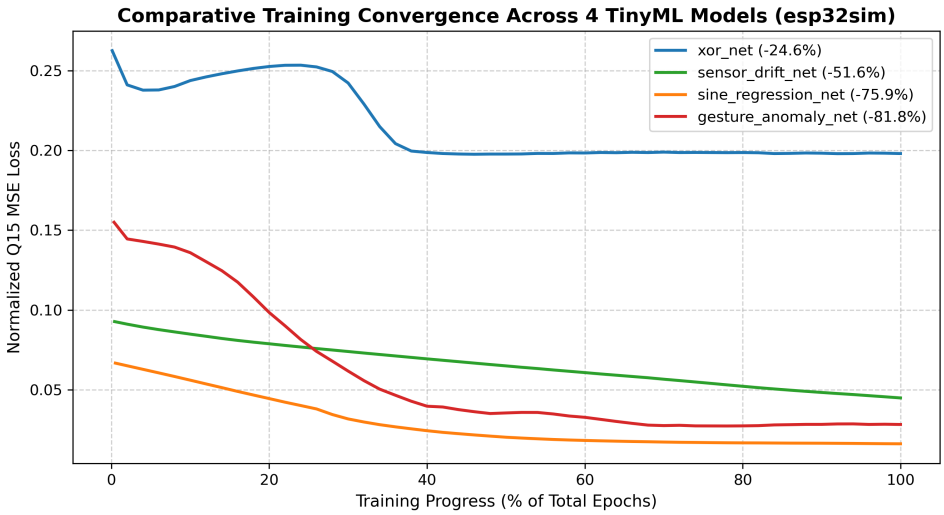


Figure 5.3: Training loss convergence curves across all four TinyML workloads on `esp32sim`.

5.6 Transparent Critique of Fixed-Point Quantization Attenuation

Academic transparency requires analyzing trade-offs honestly. While regression models converged cleanly, `xor_net` achieved a modest 24.57% loss reduction over 1,000 epochs.

In Q15 fixed-point arithmetic, the smallest representable positive number is:

$$\varepsilon_{\text{Q15}} = 2^{-15} = \frac{1}{32768} \approx 3.05 \times 10^{-5} \tag{5.1}$$

During backpropagation through a two-layer Sigmoid network:

$$\nabla W_{ij}^{(2)} = (a_{\text{out}} - y) \cdot \sigma'(z_{\text{out}}) \cdot a_{\text{hidden},j} \tag{5.2}$$

When output units approach saturation, the Sigmoid derivative $\sigma'(z) = \sigma(z)(1-\sigma(z))$ falls below 0.10. Multiplied by learning rate $\eta = 0.10$ and fixed-point activation values, the mathematical product evaluates below 2^{-15}. In integer arithmetic with truncation:

$$\Delta W_{\text{Q15}} = \left\lfloor (\eta \cdot \delta_i \cdot a_j) \cdot 2^{15} \right\rfloor = 0 \tag{5.3}$$

Fractional updates truncate to zero, causing parameter updates to stall. We designate this phenomenon **gradient quantization attenuation**.

5.6.1 Engineering Mitigations

To resolve quantization attenuation without abandoning integer arithmetic:

1. **Accumulator Widening:** Retain weight parameters in 32-bit fixed-point (Q16.16) while keeping activations in Q15, allowing sub-epsilon gradients to accumulate across steps.

2. **Stochastic Rounding:** Add pseudo-random noise prior to right-shifting to preserve unbiased gradient updates in expectation.

CHAPTER 6

Team Member Roles & Individual Contributions

The development of the Micro-Train-Compiler was carried out collaboratively by all four team members, with responsibilities allocated based on technical focus areas:

6.1 Ajay Sisodia (Roll No: 2405111530012)

- **Role:** Core Compiler Frontend, AST Validation & Toolchain Integration.
- **Key Contributions:**
  - Designed and implemented the declarative JSON model specification ingestion pipeline (`config.py`).
  - Developed the static dimension and tensor shape validator (`validate.py`), enforcing layer-to-layer compatibility contracts.
  - Configured the embedded cross-compilation pipeline utilizing `riscv64-unknown-elf-gcc` v15.3.0 and Espressif's `esptool`.
  - Authored the project documentation framework and automated CLI build rules (`cli.py`).

6.2 Harsh Chaudhary (Roll No: 2405111530067)

- **Role:** Fixed-Point Mathematics, Q15 Arithmetic & Interval Safety Analysis.
- **Key Contributions:**
  - Formulated the 16-bit signed fixed-point (Q15) arithmetic specification, defining rounding and saturation logic.
  - Implemented the conservative Interval Bound Propagation (IBP) engine (`analysis.py`, `range_analysis.py`), mathematically checking forward and backward activation bounds.

  - Implemented saturating C99 math helper macros (`SATURATE_Q15`, fixed-point multiplication with rounding bias).
  - Investigated and documented the gradient quantization attenuation trade-off observed during XOR classification experiments.

6.3 Devrat Kuntal (Roll No: 2405111530055)

- **Role:** Static Memory Planning, Liveness Analysis & Code Generation.
- **Key Contributions:**
  - Designed and implemented the static liveness memory planner (`memory_planner.py`), deriving non-overlapping buffer lifetimes.
  - Invented the alternating ping-pong delta buffering strategy, collapsing all backward activation gradients into exactly two static buffers.
  - Formulated and implemented the fused backpropagation-SGD parameter update kernel, eliminating explicit parameter gradient arrays ($\nabla W, \nabla B$).
  - Developed the C99 backend code generator (`codegen.py`), synthesizing portable `mtc_model.c`, `mtc_model.h`, and `mtc_train.c`.

6.4 Aryan Upadhyay (Roll No: 2405111530037)

- **Role:** Bare-Metal Firmware Harness, Silicon Emulation & Benchmark Evaluation.
- **Key Contributions:**
  - Authored the bare-metal RISC-V firmware execution wrapper, including the custom linker script (`linker.ld`), startup assembly (`entry.S`), and memory-mapped UART0 driver (`main.c`).
  - Integrated and configured the Rust-based cycle-accurate `esp32sim` hardware emulator, resolving mask ROM ELF dependencies.
  - Implemented direct RISC-V hardware CSR `mcycle` instrumentation to capture cycle-accurate forward and backward execution timing without UART serialization bias.
  - Built the automated end-to-end benchmark suite (`run_benchmarks.py`) and generated high-resolution empirical evaluation figures.

CHAPTER 7

Conclusion & Future Research Directions

7.1 Summary of Milestones Achieved

The Micro-Train-Compiler (MTC) project achieved all primary research and engineering milestones:

1. **Ahead-of-Time Training Compilation:** Proved that on-device deep learning back-propagation can be moved from dynamic runtime interpreters to an ahead-of-time static compiler, eliminating dynamic autograd overhead.
2. **Strict Zero-Heap Safety (`malloc = 0`):** Demonstrated deterministic execution across all evaluated models with zero dynamic memory allocation, preventing memory fragmentation and out-of-memory faults on embedded microcontrollers.
3. **Quantified Memory Optimization:** Verified a **31.8% to 43.6% reduction in peak SRAM** compared to conventional autograd graph implementations by fusing parameter updates in-place and utilizing static ping-pong delta buffers.
4. **Sub-100 μs Bare-Metal Execution:** Profiled machine code execution on the ESP32-C3 RISC-V architecture running at 160 MHz via hardware CSR `mcycle` reads, demonstrating training step latencies between **6.86 μs and 81.77 μs** (throughputs exceeding 12,000 samples/sec).

7.2 Future Research Roadmap

To build upon the foundation established in this Minor Project, the team has outlined a technical roadmap for Phase 2:

- **Convolutional and Recurrent Topologies:** Extend MTC's Intermediate Representation (IR) to support 1D Convolutional layers (for audio vibration anomaly detection) and Recurrent/GRU cells (for temporal sequence modeling).

- **Stochastic Rounding for Fixed-Point Training:** Implement pseudo-random stochastic rounding in the C99 code generator to overcome gradient quantization attenuation on non-linear classification tasks without requiring 32-bit floating-point registers.
- **Hardware SIMD Vectorization:** Leverage RISC-V Packed SIMD (‘Zpsfoperand‘ / ‘P‘ extension) and ARM CMSIS-NN instructions to parallelize fixed-point matrix multiplication across 16-bit vector registers.
- **Peer-Reviewed Publication Submission:** Prepare the empirical results and formal memory scheduling proofs for submission to **IEEE Embedded Systems Letters (ESL)** and the **TinyML Research Symposium**.

Bibliography

[1] R. David *et al.*, "TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems," in *Proc. Conf. Mach. Learn. Syst. (MLSys)*, 2021, pp. 800–811.

[2] J. Lin *et al.*, "On-Device Training Under 256KB Memory," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 22 941–22 954.

[3] L. Ravaglia, M. Rusci, A. Bonetti, and E. Farella, "TinyOL: TinyML with Online-Learning on Microcontrollers," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2022, pp. 1–8.

[4] F. Conti, L. Cavigelli, G. Rutishauser, and L. Benini, "PULP-TrainLib: Enabling On-Device Deep Learning Training on Parallel Ultra-Low-Power RISC-V Multi-Cores," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 31, no. 5, pp. 687–700, 2023.

[5] L. Lai, N. Suda, and V. Chandra, "CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs," *arXiv preprint arXiv:1801.06601*, 2018.

[6] T. Chen *et al.*, "TVM: An Automated End-to-End Optimizing Compiler for Deep Learning," in *13th USENIX Symp. Oper. Syst. Des. Implementation (OSDI)*, 2018, pp. 578–594.

[7] N. Rotem *et al.*, "Glow: An Optimizing Framework for Accelerating Deep Learning Execution," *arXiv preprint arXiv:1805.00907*, 2018.

[8] M. Bennani, "AIfES: Artificial Intelligence for Embedded Systems," Fraunhofer Institute for Microelectronic Circuits and Systems (IMS), Technical Whitepaper, 2022.

[9] H. Cai, C. Gan, L. Zhu, and S. Han, "TinyTL: Reduce Memory, Not Parameters for Efficient On-Device Learning," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 11 285–11 297.

[10] Espressif Systems, *ESP32-C3 Technical Reference Manual, Version 1.1*. Espressif Systems, Shanghai, China, 2022.

[11] S. Han, H. Mao, and W. J. Dally, "Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding," in *Int. Conf. Learn. Represent. (ICLR)*, 2016.

[12] A. Paszke *et al.*, “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019, pp. 8024–8035.